

Documento de Planeación Técnica

Arquitectura SaaS escolar multi-tenant con app nativa, subdominios por colegio y backend centralizado

Versión	v1.1 - ampliada
Uso previsto	Documento de trabajo para desarrollador asociado y planeación de implementación
Alcance	Planeación de producto, requerimientos, arquitectura y criterios para iniciar desarrollo.
Nota	Documento base para revisión con desarrolladores, socios y colegios interesados. No reemplaza asesoría legal ni levantamiento formal definitivo.

Preparado para discusión técnica y funcional. Los nombres de marca, precios y módulos pueden ajustarse según estrategia comercial.

Tabla de contenido

- 1. Resumen de arquitectura**
- 2. Stack recomendado**
- 3. Monorepo y estructura**
- 4. Modelo multi-tenant**
- 5. Dominios y resolución de tenant**
- 6. Base de datos**
- 7. Backend/API**
- 8. App nativa**
- 9. Seguridad**
- 10. Integraciones**
- 11. Infraestructura**
- 12. CI/CD y ambientes**
- 13. Backups y monitoreo**
- 14. Roadmap técnico**
- 15. Riesgos y decisiones**

La numeración de páginas puede actualizarse si este documento se convierte posteriormente a DOCX editable con tabla de contenido automática.

1. Resumen técnico ejecutivo

La solución se diseñará como una plataforma SaaS multi-tenant. El mismo backend y la misma infraestructura base servirán a múltiples colegios, separando datos y configuración mediante tenant_id, resolución por dominio/subdominio y controles estrictos de autorización.

Componentes principales:

- Landing comercial: `www.dominio.co`
- Panel interno SaaS: `admin.dominio.co`
- API central: `api.dominio.co`
- App web por colegio: `app.{colegio}.dominio.co`
- App nativa: Android/iOS conectada a la misma API
- Base de datos: PostgreSQL
- Cache/colas: Redis + BullMQ
- Storage: S3/R2 compatible
- Backend: NestJS + Prisma
- Frontend web: Next.js + TypeScript
- Mobile: React Native + Expo

Decisión técnica principal

Construir multi-tenant desde el día uno. Aunque los primeros clientes sean pocos, adaptar luego un sistema single-tenant a SaaS suele ser más costoso y riesgoso.

2. Arquitectura lógica

`www.dominio.co`
-> Landing comercial, captación de leads y solicitud de demo

`admin.dominio.co`
-> Panel interno del operador SaaS

`app.colegio.dominio.co`
-> Aplicación web del colegio / tenant

`api.dominio.co`
-> API central para web, mobile, integraciones y jobs

Mobile App
-> React Native/Expo consumiendo `api.dominio.co` con tenant resuelto por colegio/código/dominio

La API debe ser centralizada. No se debe desplegar un backend diferente para cada colegio en la primera etapa, salvo en planes Enterprise futuros.

3. Stack recomendado

Capa	Tecnología	Motivo
------	------------	--------

Web	Next.js + TypeScript	Permite landing, paneles privados, routing, SSR/SEO en sitio comercial y buen ecosistema React.
UI	Tailwind CSS + componentes propios/shadcn	Rapidez visual, diseño consistente y reutilización.
Mobile	React Native + Expo	App nativa multiplataforma usando React/TypeScript.
Backend	NestJS + TypeScript	Arquitectura modular, inyección de dependencias, guards, pipes y escalabilidad.
ORM	Prisma	Migraciones, tipos TS y productividad.
DB	PostgreSQL	Relacional, robusta, buena para datos académicos y reportes.
Jobs	Redis + BullMQ	Reportes, correos, notificaciones, importaciones y procesos pesados.
Storage	Cloudflare R2/AWS S3 compatible	Archivos externos a la base de datos, URLs firmadas.
Infra	Docker + VPS/cloud	Portabilidad, despliegue controlado y escalamiento gradual.

4. Estructura de monorepo

Se recomienda monorepo con pnpm workspaces para compartir tipos, validaciones y utilidades entre web, mobile y API.

```

school-saas/
├── apps/
│   ├── web/          # Next.js: landing, panel SaaS, panel colegio
│   ├── mobile/       # React Native + Expo
│   └── api/           # NestJS backend
├── packages/
│   ├── database/     # Prisma schema, migrations, seeders
│   ├── shared/       # Tipos compartidos, constantes, helpers
│   ├── validators/   # Zod schemas reutilizables
│   ├── ui/           # Componentes UI compartidos web
│   └── config/        # ESLint, TSConfig, Tailwind config
├── docker/
├── docs/
├── scripts/
├── .github/workflows/
└── README.md

```

- Evitar duplicar tipos entre frontend y backend.

- Compartir validaciones de formularios cuando sea posible.
- Mantener paquetes desacoplados: mobile no debe depender de componentes web.
- Versionar migraciones y seeds demo.
- Documentar comandos de desarrollo desde el inicio.

5. Modelo multi-tenant

La primera versión debe usar una base de datos compartida con separación lógica por tenant_id. Esto mantiene costos bajos y simplifica actualizaciones para varios colegios.

Estrategia	Uso recomendado	Ventajas	Desventajas
DB compartida + tenant_id	Primera versión y planes estándar.	Menor costo, más simple de operar, actualizaciones centralizadas.	Requiere disciplina estricta en cada consulta.
Schema por tenant	Escalamiento intermedio si aumenta complejidad.	Más aislamiento lógico.	Migraciones más complejas.
DB dedicada por tenant	Plan Enterprise.	Aislamiento fuerte y acuerdos especiales.	Mayor costo y operación más compleja.

Regla técnica obligatoria

Toda tabla que contenga datos de colegio debe incluir tenant_id, salvo catálogos globales controlados como planes o permisos base.

6. Resolución de tenant por dominio

El formato solicitado es app.colegio.dominio.co. El sistema debe extraer el slug del colegio desde el hostname, validarlo contra la tabla tenants y cargar configuración correspondiente.

Ejemplo:

Hostname recibido: app.sanpedro.dominio.co

Partes: app | sanpedro | dominio | co

Tenant slug detectado: sanpedro

Consulta:

```
SELECT * FROM tenants
```

```
WHERE slug = 'sanpedro'
```

```
AND status IN ('active', 'pilot', 'demo');
```

Dominio	Función	Tenant
www.dominio.co	Landing comercial.	No aplica
admin.dominio.co	Panel interno SaaS.	No aplica / operador

api.dominio.co	API central.	Se resuelve por header, token o request context.
app.sanpedro.dominio.co	Panel/app web del colegio San Pedro.	sanpedro
app.demo.dominio.co	Demo con datos ficticios.	demo

6.1 DNS y SSL para subdominios de segundo nivel

El patrón app.colegio.dominio.co es visualmente profesional, pero requiere planear DNS y certificados. Un wildcard común *.dominio.co cubre subdominios como sanpedro.dominio.co, pero no siempre cubre app.sanpedro.dominio.co. Para ese patrón puede requerirse wildcard por colegio o un proveedor/proxy que permita automatizar certificados por hostname.

Alternativa	Ejemplo	Complejidad	Recomendación
Subdominio simple por colegio	sanpedro.dominio.co	Baja	Más simple técnicamente.
Subdominio doble solicitado	app.sanpedro.dominio.co	Media/Alta	Usarlo si se automatiza DNS/SSL.
Ruta por colegio	app.dominio.co/sanpedro	Baja	Menos elegante, útil como fallback.
Dominio propio del colegio	app.colegio.edu.co	Alta	Add-on o Enterprise.

7. Tabla tenants

```

tenants
- id UUID PK
- name VARCHAR
- slug VARCHAR UNIQUE
- legal_name VARCHAR NULL
- nit VARCHAR NULL
- status ENUM('demo','pilot','active','suspended','cancelled')
- plan_id UUID
- primary_domain VARCHAR
- allowed_domains TEXT[]
- logo_url TEXT NULL
- brand_color VARCHAR NULL
- timezone VARCHAR DEFAULT 'America/Bogota'
- metadata JSONB
- created_at TIMESTAMP
- updated_at TIMESTAMP

```

El campo allowed_domains permite soportar varios dominios por colegio: app.sanpedro.dominio.co, sanpedro.dominio.co y eventualmente app.colegiosanpedro.edu.co.

8. Modelo de datos base

Las entidades principales deben diseñarse para soportar colegios con varias sedes, grados, grupos, periodos y reglas de calificación.

Grupo de tablas	Tablas sugeridas
SaaS	tenants, plans, tenant_settings, tenant_domains, billing_subscriptions
Auth	users, sessions, refresh_tokens, roles, permissions, user_roles
Institución	campuses, academic_years, terms, grades, groups, classrooms
Personas	students, guardians, student_guardians, teachers, staff_profiles
Académico	subjects, teacher_subject_groups, schedules, mark_categories, assignments, marks
Asistencia	attendance_sessions, attendance_records, excuses
Tareas	homework, homework_attachments, homework_submissions
Comunicaciones	announcements, announcement_recipients, message_threads, messages
Notificaciones	notifications, push_tokens, notification_preferences
Integraciones	email_integrations, email_logs, integration_events
Archivos	files, file_access_logs
Auditoría	audit_logs, support_access_logs
Soporte	support_tickets, support_ticket_messages

9. Índices y consistencia por tenant

- Crear índices compuestos frecuentes: (tenant_id, id), (tenant_id, email), (tenant_id, group_id), (tenant_id, created_at).
- Evitar identificadores únicos globales cuando el dato puede repetirse entre colegios. Por ejemplo: código de estudiante puede ser único por tenant, no necesariamente global.
- Usar UUID para IDs públicos y evitar IDs incrementales expuestos.
- Implementar soft delete en entidades críticas como estudiantes, profesores, acudientes y notas.
- Agregar created_by, updated_by o audit logs para cambios relevantes.

Ejemplo de unicidad por tenant:

```
UNIQUE (tenant_id, document_number)
UNIQUE (tenant_id, email)
UNIQUE (tenant_id, slug)
```

10. Backend NestJS: módulos

```
src/
├─ auth/
├─ tenants/
├─ users/
├─ roles/
├─ permissions/
├─ campuses/
├─ academic-years/
├─ students/
├─ guardians/
├─ teachers/
├─ groups/
├─ subjects/
├─ schedules/
├─ attendance/
├─ marks/
├─ homework/
├─ announcements/
├─ messages/
├─ notifications/
├─ email-integrations/
├─ files/
├─ reports/
├─ audit/
├─ support/
└─ common/
```

- common debe incluir guards, decorators, interceptors, pipes, filtros de error y utilidades compartidas.
- Cada módulo debe recibir tenant context; no debe consultar datos de colegio sin tenant.
- Los servicios deben preferir métodos explícitos: findByTenant, createForTenant, updateForTenant.
- Los controladores deben validar permisos por endpoint.

11. Tenant context en backend

El tenant context debe resolverse antes de entrar a controladores protegidos. Debe poder venir del hostname, del token del usuario o de un header interno controlado para app móvil.

Flujo request web:

1. Request llega con Host: app.sanpedro.dominio.co
2. TenantResolver extrae slug = sanpedro
3. Busca tenant activo
4. Adjunta currentTenant al request
5. Auth valida usuario

6. Guard valida que user.tenant_id coincide con currentTenant.id

7. Controller ejecuta acción con tenant_id obligatorio

App móvil y tenant

En app móvil no siempre existe hostname por colegio. Puede resolverse con código de colegio, dominio seleccionado en onboarding o tenant_slug guardado después del primer login.

12. API: convenciones

Convención	Recomendación
Formato	REST JSON inicial. GraphQL no es necesario al inicio.
Versionado	/v1/... para permitir cambios futuros.
Errores	Estructura estándar: code, message, details, requestId.
Paginación	page/pageSize o cursor en listados grandes.
Validación	DTOs + Zod/class-validator.
Auditoría	Interceptor o servicio explícito en acciones críticas.
Idempotencia	Usar claves en importaciones, jobs y notificaciones para evitar duplicados.

Ejemplos de endpoints:

POST /v1/auth/login

POST /v1/auth/refresh

GET /v1/me

GET /v1/tenants/current

GET /v1/students

POST /v1/students/import

GET /v1/teacher/today

POST /v1/attendance/sessions/:id/records

POST /v1/announcements

POST /v1/mobile/push-token

GET /v1/guardian/children/:childId/grades

13. Autenticación

- Login por email y contraseña.
- Access token de corta duración.
- Refresh token rotativo.
- Sesiones por dispositivo.
- Revocación de sesiones.

- Recuperación de contraseña por correo.
- 2FA para superadministradores, soporte y administradores de colegio.
- Bloqueo temporal ante intentos fallidos repetidos.
- Registro de IP, dispositivo y fecha de acceso.

13.1 Tokens en app móvil

- Guardar refresh token en Secure Store.
- Access token preferiblemente en memoria o almacenamiento seguro si se requiere persistencia.
- Registrar device_id para asociar push token.
- Permitir cierre remoto de sesión desde el panel.
- Biometría opcional para desbloqueo local, no como único método de autenticación.

14. Autorización y permisos

El sistema debe implementar RBAC con permisos granulares. Un rol puede tener varios permisos y un usuario puede tener roles según su tenant.

Roles base:

- super_admin
- support_agent
- tenant_admin
- principal
- coordinator
- secretary
- teacher
- guardian
- student

Permisos ejemplo:

- students.read
- students.create
- students.update
- attendance.create
- attendance.update
- marks.create
- marks.publish
- announcements.create
- reports.export
- settings.update

Regla de autorización

No basta con que el usuario esté autenticado. Cada acción debe validar tenant, rol, permiso y alcance de datos. Por ejemplo,

15. App nativa: arquitectura

Área	Recomendación
Framework	React Native + Expo.
Routing	Expo Router.
Estado remoto	TanStack Query.
Estado local simple	Zustand o Context API si es suficiente.
Formularios	React Hook Form + Zod.
Notificaciones	Expo Notifications / FCM según build.
Storage seguro	Expo Secure Store.
Archivos/cámara	Expo ImagePicker, DocumentPicker según necesidad.
Build	EAS Build para Android/iOS.

La app debe estar organizada por rol, pero sin crear tres aplicaciones separadas. El login identifica el rol y habilita navegación correspondiente.

```
apps/mobile/
├─ app/
│   ├─ (auth)/
│   ├─ (guardian)/
│   ├─ (teacher)/
│   ├─ (student)/
│   └─ settings/
├─ src/
│   ├─ api/
│   ├─ components/
│   ├─ features/
│   ├─ hooks/
│   ├─ storage/
│   └─ utils/
```

16. Flujo de onboarding en app móvil

1. Usuario instala la app.
2. Selecciona colegio mediante código, búsqueda o enlace de invitación.
3. La app guarda tenant_slug o tenant_id público.
4. Usuario inicia sesión con credenciales.

5. API valida tenant + usuario + rol.

6. App registra push token.

7. App muestra home según rol.

8. Usuario puede cambiar de colegio si su cuenta pertenece a más de un tenant, caso poco común pero posible.

17. Notificaciones push: diseño técnico

```
push_tokens
- id UUID
- tenant_id UUID
- user_id UUID
- device_id VARCHAR
- platform ENUM('ios','android')
- token TEXT
- active BOOLEAN
- last_seen_at TIMESTAMP
- created_at TIMESTAMP
- updated_at TIMESTAMP
```

- Los eventos de negocio generan registros de notificación.
- Un worker procesa la cola y envía push.
- Registrar éxito, error y reintentos.
- Respetar preferencias por usuario y configuración por colegio.
- No incluir datos sensibles completos en el payload.
- Invalidar tokens rechazados por el proveedor.

18. Archivos y storage

Los archivos no deben guardarse como binarios en PostgreSQL. Deben ir a storage compatible S3 y la base de datos debe almacenar metadata.

```
Ruta sugerida:
/tenants/{tenant_id}/students/{student_id}/documents/{file_id}
/tenants/{tenant_id}/homework/{homework_id}/attachments/{file_id}
/tenants/{tenant_id}/submissions/{submission_id}/{file_id}
/tenants/{tenant_id}/announcements/{announcement_id}/{file_id}

files
- id
- tenant_id
- owner_type
- owner_id
```

- file_name
- mime_type
- size_bytes
- storage_key
- uploaded_by
- visibility
- created_at

- Usar URLs firmadas para descarga.
- Validar tipo y tamaño de archivo.
- Escanear o restringir tipos peligrosos si se escala.
- Separar archivos por tenant.
- Auditar descargas de documentos sensibles.

19. Correos institucionales e integraciones

El sistema debe permitir integración por tenant, sin convertirse en un proveedor de correo o webmail.

Tipo integración	Uso	Complejidad
Proveedor transaccional autorizado	Notificaciones desde remitente del colegio con DNS configurado.	Media
SMTP institucional	Envíos usando cuenta institucional técnica.	Media
Google Workspace	OAuth/API para envío avanzado y quizá directorio.	Alta
Microsoft 365	Microsoft Graph para envío avanzado y quizá directorio.	Alta

```
email_integrations
- id
- tenant_id
- provider ENUM('transactional','smtp','google_workspace','microsoft_365')
- sender_email
- sender_name
- reply_to
- status ENUM('pending','active','error','disabled')
- encrypted_credentials JSONB
- last_test_at
- created_at
- updated_at
```

20. Importación masiva de datos

La importación es crítica para onboarding. Debe ser robusta y segura.

1. Usuario descarga plantilla oficial.
 2. Usuario carga CSV/XLSX.
 3. Backend guarda archivo temporal.
 4. Worker analiza columnas y valida filas.
 5. Sistema muestra previsualización con errores y advertencias.
 6. Usuario confirma importación.
 7. Sistema ejecuta transacción o proceso con rollback controlado.
 8. Se genera reporte de importación.
 9. Se registra auditoría.
- No aplicar cambios directamente sin previsualización.
 - Detectar duplicados por documento/correo dentro del tenant.
 - Permitir corrección de errores antes de confirmar.
 - Usar jobs para importaciones grandes.
 - Mantener historial de importaciones.

21. Reportes asíncronos

Los reportes grandes no deben bloquear la API. Se deben generar por jobs.

Flujo:

1. POST /reports/attendance
2. Crear report_job con status queued
3. Worker genera PDF/Excel
4. Guardar archivo en storage
5. Actualizar status completed
6. Notificar al usuario
7. Usuario descarga con URL firmada

Reporte	Estrategia
Boletín individual	Puede ser síncrono si es rápido; preferible job en lote.
Consolidado de notas	Job asíncrono.
Asistencia por periodo	Job asíncrono.
Comunicados y lecturas	Job asíncrono.

22. Auditoría técnica

```
audit_logs
- id UUID
- tenant_id UUID NULL
- user_id UUID NULL
- actor_role VARCHAR
- action VARCHAR
- entity_type VARCHAR
- entity_id UUID NULL
- old_values JSONB NULL
- new_values JSONB NULL
- ip_address VARCHAR
- user_agent TEXT
- request_id VARCHAR
- created_at TIMESTAMP
```

- No registrar contraseñas ni secretos.
- En cambios sensibles guardar old_values/new_values resumidos.
- Auditar acceso de soporte a tenants.
- Permitir filtros por usuario, acción, fecha y entidad.
- Definir retención de logs según plan y política.

23. Seguridad

Área	Control recomendado
Contraseñas	Hash fuerte con algoritmo moderno, nunca texto plano.
Tokens	Access corto + refresh rotativo. Revocación por dispositivo.
Tenant isolation	Guards, filtros por tenant_id y tests automatizados.
Archivos	URLs firmadas, validación de tipo/tamaño, rutas por tenant.
Datos sensibles	Minimizar datos en push/correos y cifrar secretos.
Administradores	2FA y registro de actividad.
Soporte	Acceso temporal con auditoría e idealmente aprobación.
API	Rate limiting, CORS controlado, validación DTO y protección de endpoints.

Pruebas de aislamiento

Debe existir una batería de tests que intente acceder a datos de otro tenant y confirme que la API responde 403/404 o no retorna resultados.

24. Ambientes

Ambiente	Uso	Datos
development	Desarrollo local.	Ficticios/locales.
staging	Pruebas internas antes de producción.	Ficticios o anonimizados.
demo	Demostraciones comerciales.	Ficticios reseteables.
production	Clientes reales.	Reales, protegidos y respaldados.

25. Infraestructura inicial

Para los primeros colegios se puede iniciar con infraestructura moderada, pero separando al menos aplicación y base de datos si el presupuesto lo permite.

Servicio	Recomendación inicial	Notas
Servidor app	4-8 vCPU, 8-16 GB RAM	Web, API, workers livianos.
Servidor DB	4 vCPU, 8-16 GB RAM	PostgreSQL y Redis si no se separa aún.
Storage	S3/R2 externo	Archivos, reportes, adjuntos.
Backups	Externo al servidor principal	Restauración probada periódicamente.
CDN/proxy	Opcional al inicio	Útil para dominios, SSL y assets.
Monitoreo	Desde el día uno	Uptime, errores, métricas, logs.

26. Docker y despliegue

Servicios sugeridos en docker-compose inicial:

- web
- api
- worker
- postgres
- redis
- nginx/traefik/caddy

Servicios externos:

- storage S3/R2
- correo transaccional/SMTP

- push notifications
- monitoreo

- Usar imágenes separadas para web, api y worker.
- No ejecutar migraciones automáticamente sin control en producción.
- Usar variables de entorno por ambiente.
- Rotar secretos si se filtran.
- Mantener staging lo más parecido posible a producción.

27. CI/CD

1. Instalar dependencias con pnpm.
2. Ejecutar lint.
3. Ejecutar typecheck.
4. Ejecutar tests unitarios.
5. Ejecutar tests de aislamiento multi-tenant.
6. Construir web, api y mobile si aplica.
7. Crear imagen Docker.
8. Desplegar a staging.
9. Ejecutar migraciones controladas.
10. Desplegar a producción con aprobación manual.

28. Backups y recuperación

Elemento	Frecuencia sugerida	Retención
PostgreSQL	Diario automático + antes de migraciones críticas.	7 diarios, 4 semanales, 6-12 mensuales.
Storage	Versionado o backup periódico según proveedor.	Según plan.
Variables/configuración	Cada cambio importante.	Histórico seguro.
Demo	Seed reproducible, no requiere backup fuerte.	Recreable.

Regla de backup

Un backup no probado no cuenta como backup. Se deben programar pruebas de restauración periódicas, aunque sean manuales al inicio.

29. Monitoreo y observabilidad

- Disponibilidad de web, API y app endpoints.
- Errores backend con stack trace y requestId.
- Errores frontend y mobile.
- Tiempo de respuesta por endpoint.
- CPU, RAM, disco y conexiones DB.
- Jobs fallidos o en cola demasiado tiempo.
- Correos y notificaciones fallidas.
- Uso por tenant: usuarios activos, storage, reportes, notificaciones.
- Alertas ante caídas o errores críticos.

30. Roadmap técnico sugerido

Fase	Objetivo	Entregables
1. Base	Fundación del SaaS.	Monorepo, auth, tenants, dominios, roles, panel interno básico.
2. Académico	Estructura escolar.	Estudiantes, acudientes, profesores, grados, grupos, materias, horarios.
3. Operación	Uso diario.	Asistencia, notas, tareas, comunicados, calendario, reportes.
4. Mobile	App nativa profesional.	Padres, profesores, estudiantes, push, excusas, tareas, notas.
5. Integraciones	Comunicación formal.	Correo institucional, logs, plantillas, configuración por tenant.
6. Producción	Operación estable.	Backups, monitoreo, demo, soporte, documentación, piloto.

31. Criterios técnicos de listo

1. Todas las tablas críticas tienen tenant_id.
2. Los endpoints validan tenant y permisos.
3. Existe resolución de tenant por hostname.
4. Existe seed de demo reproducible.

5. La app móvil puede seleccionar/recordar colegio y autenticar usuario.
6. Push tokens se registran y pueden recibir notificaciones.
7. Los archivos se guardan en storage externo con metadata en DB.
8. Los reportes pesados corren como jobs.
9. Hay auditoría en acciones críticas.
10. Hay backups automáticos y prueba de restauración documentada.
11. Hay monitoreo básico y manejo de errores con requestId.
12. Hay pruebas de aislamiento multi-tenant.

32. Riesgos técnicos y mitigaciones

Riesgo	Impacto	Mitigación
Fuga de datos entre tenants	Crítico	tenant_id obligatorio, guards, tests, auditoría y revisión de queries.
Complejidad DNS/SSL con app.colegio.dominio.co	Alto	Automatizar dominios/certificados o usar fallback sanpedro.dominio.co.
App móvil muy grande desde el inicio	Alto	Priorizar flujos por rol y construir módulos críticos primero.
Importaciones inestables	Alto	Validación previa, jobs, transacciones y reportes de error.
Cálculo de notas rígido	Medio/Alto	Diseñar configuración flexible por tenant.
Reportes lentos	Medio	Generación asíncrona y cache cuando aplique.
Soporte sin trazabilidad	Medio/Alto	Tickets, logs e impersonation auditada.

33. Decisiones abiertas para validar con el equipo

- Nombre final del producto y dominio disponible.
- Proveedor de DNS/proxy/SSL para soportar app.colegio.dominio.co.
- Si la primera app se publicará bajo marca propia del SaaS o marca blanca.
- Nivel inicial de integración con correo: transaccional, SMTP, Google Workspace o Microsoft 365.
- Profundidad del módulo de tareas frente a integración futura con Moodle/LMS.
- Alcance de reportes PDF personalizados en primera versión.
- Modelo de precios y límites por plan.

- Procedimiento de soporte y SLA por plan.

34. Recomendación técnica final

La solución debe comenzar con una base SaaS robusta: tenants, dominios, roles, usuarios, datos académicos, app nativa, notificaciones, auditoría y reportes. La tentación de construir muchos módulos secundarios debe controlarse para evitar retrasar la salida comercial.

Arquitectura recomendada

Next.js + NestJS + PostgreSQL + Prisma + Redis/BullMQ + React Native/Expo + S3/R2 + Docker, con monorepo pnpm y separación estricta por tenant.